Compte-rendu de TP – Chiffrement 1

GEII – INFO1 – TD/TP Chiffrement n°1 – Édition 2024

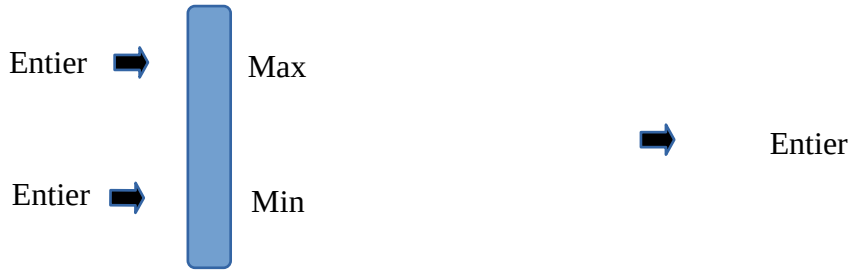
Nom : *FRILAY Adam / SAHAOUI Marouane*
Date : *21/10/2025*
Encadrant : ILLIEN Francois, LEVILLY Sébastien

I. Vérification et mise au point (Programme principal et actions paramétrées)

1. Exigences fonctionnelles identifiées

- **Programme principal :**
  Assurer la bonne exécution des différentes actions paramétrées liées au chiffrement et déchiffrement, à l'exception de `dechiffrerCar` non implémentée.

- **Actions paramétrées fournies :**



- `acquerirCle( min: Entier, max : Entier) :`
  La fonction acquerirCle retourne un entier. Elle est là pour acquerie la Clé et empêcher une valeur qui n'est pas dans l'intervalle.

- `char chiffrerCar(Car : caractère, Cle: entier) :`
  La fonction chiffrerCAr retourne un caractère. Sa fonctionnaltié est de trier les caractères maj et mini pour les envoyer dans la bonne action/le bon chiffrement.

- `char chiffrerMaj(Car : caractère, Cle: entier)` : La fonction chiffrerMaj retourne un caractère. Elle sert à chiffrer les caractères majuscules avec la clé rentrée par l'utilisateur.

- `char chiffrerMin(Car : caractère, Cle: entier)` : La fonction chiffrerMin retourne un caractère. Elle sert à chiffrer les caractères minuscule avec la clé rentrée par l'utilisateur.

- `char dechiffrerCar (Car : caractère, Cle: entier)` : La fonction dechiffrerCar retourne un caractère. Sa fonctionnalité est de dechiffrer le chiffrement et donc de donner les caractères en entrée (Chemin inverse)

Comportement attendu :

- Validation des entrées utilisateur.
- Chiffrement circulaire respectant les bornes alphabétiques.
- Production de résultats conformes au cahier des charges.

Fiche de tests TP2 - Programme Chiffrement

☞ *Si besoin, on pourra insérer des captures d'écran pour mieux illustrer les résultats.*

1.Fiche de test sur `acquerirCle`

1. Fonctionnalité testée :	On veut vérifier que la saisie s'arrête quand l'utilisateur saisit '.'
2. Scénario Donnée(s) d'entrée : '.' Situation(s) visée(s) :	Le programme reconnaît la saisie de '.' et s'arrête.
3. Résultat attendu :	Le programme s'arrête à la saisie du '.'
4. Résultat obtenu :	On remarque que le point ne peut pas être saisie directement mais seulement lorsque le nombre de caractère saisie dépasse 10. Celà est dû à la condition dans le while. `} while ((unCar != '.') || (nbCar < MAX_NB_CAR));`
5. Conclusion :	Il faut changer la condition du while en remplaçant | | par && pour que l'on puisse entrer un point quand l'on veut. Après modification du programme, le point peut être saisie n'importe quand et arrête le programme.

1. Fonctionnalité testée :	On veut vérifier que la saisie s'arrête quand l'utilisateur saisit + de 80 caractère
2. Scénario Donnée(s) d'entrée : A B D Z R → jusqu'à 80 caractère Situation(s) visée(s) :	Le programme reconnaît quand + de 80 caractère sont entrée et s'arrête
3. Résultat attendu :	Le programme s'arrête quand il atteint 80 caractère
4. Résultat obtenu :	Le programme s'arrête à 10 caractères entrée

5. Conclusion :	Le programme s'arrête seulement à 10 caractère entrée car la constante MAX_NB_CAR est définit à 10. Si on change cette constante à 80, le programme marche et s'arrête lorsque la saisie atteint 80 caractères.

2. Action paramétrée `acquerirCle()`

1. Fonctionnalité testée :	On veut vérifier qu'un message « Erreur » s'affiche lorsque la valeur de la clé n'est pas dans l'intervalle donnée. Et que le programme ne se lance pas.
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 - 4 L'action paramétrée acquerirCle reconnaît que la valeur n'est pas dans l'intervalle
3. Résultat attendu :	« Erreur » + le programme s'arrête.
4. Résultat obtenu :	

```
Quelle valeur pour la cle (entre 0 et 25) ? : 26

ERREUR !!!
Fournir les caracteres a chiffrer...


Car     | Car   | Car
Init    | Code  | Decode
1       |       |
5       |       |
1       |       |
4       |       |
```

5. Conclusion :	Le message d'erreur s'affiche mais le programme continue. Il faut donc inclure une acquisition contrôlée pour forcer l'utilisateur à entrée une cle dans l'intervalle donné.

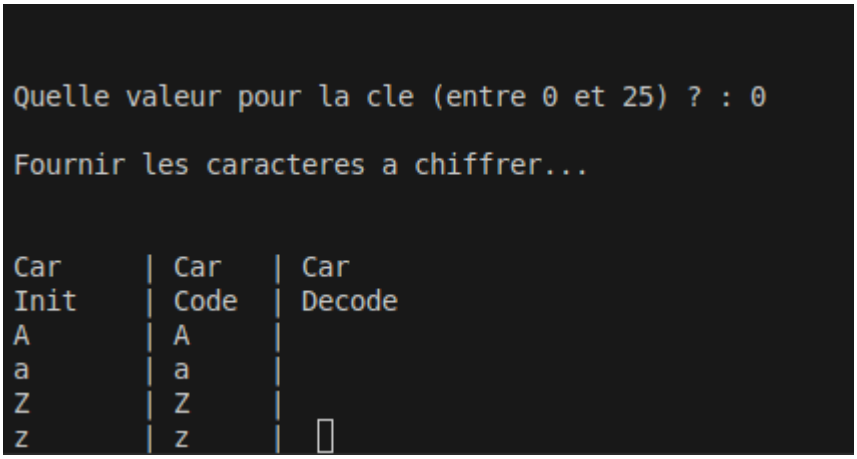
On à donc ajouté cette acquisition contrôlé :

```
int valLue;
 do
{
    printf("\nQuelle valeur pour la cle (entre %d et %d) ?
    scanf("%d", &valLue);
    if ((valLue < min) || (valLue > max))
    {
        printf("\nERREUR !!!");
    }
} while ((valLue < min) || (valLue > max));
return valLue;
```

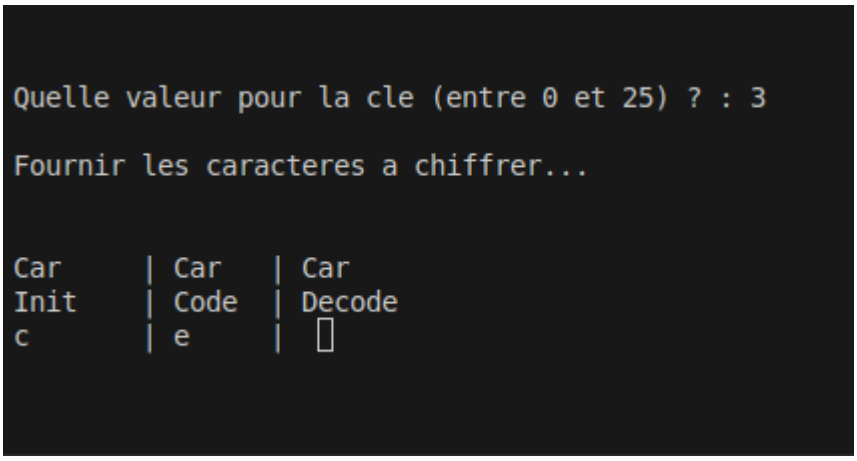
```
Quelle valeur pour la cle (entre 0 et 25) ? : -8

ERREUR !!!
Quelle valeur pour la cle (entre 0 et 25) ? : 98

ERREUR !!!
Quelle valeur pour la cle (entre 0 et 25) ? : 26

ERREUR !!!
Quelle valeur pour la cle (entre 0 et 25) ? : -4

ERREUR !!!
Quelle valeur pour la cle (entre 0 et 25) ? :
```

3. Action paramétrée `chiffrerCar()`

1. Fonctionnalité testée :	On cherche à vérifier que la fonction fait bien la différence entre les caractère majuscule et minuscule.
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 A → a → Z →z L'action paramétrée reconnaît quand les lettres sont majuscules et minuscles et les chiffre de la bonne manière.
3. Résultat attendu :	Le programme avance prochain d'entrée le prochain caractère

4. Résultat obtenu :	
5. Conclusion :	L'action paramétrée ChiffrerCar fait la différence entre les caractère majuscule et minuscules

4. Action paramétrée `chiffrerMin()`

1. Fonctionnalité testée :	Vérifier que le décalage donné par la clef est bien respectée par l'action paramétrès
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 c avec une clé = 3 Le programme décale bien la lettre à l'aide de la table ascii par la valeur de cle
3. Résultat attendu :	C → e
4. Résultat obtenu :	

5. Conclusion :	Le programme décale bien la lettre à l'aide de la table ascii par la valeur de cle tout en restant dans les minuscule

1. Fonctionnalité testée :	Vérifier qu'aucun caractère codé aient qu'un seul caractère initial afin de faciliter le décodage
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 On vient entrer l'alphabet Aucune lettre codé à deux caractère initial
3. Résultat attendu :	Chaque lettre à sa propre
4. Résultat obtenu :	Chaque char Code à un seul et unique char initial
5. Conclusion :	Aucun caractère code à deux caractère initiaux nous permettant le codage.

5. Action paramétrée `chiffreMaj()`

1. Fonctionnalité testée :	Vérifier que le chiffrement des Majuscule reste une Majuscule comme dans le chiffrement de César
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 Z avec une clé = 25 Vérifier que le décale est bien respectée
3. Résultat attendu :	X
4. Résultat obtenu :	s

5. Conclusion :	Le programme ne revient pas à A majuscule il faut donc lorsque Z est frappé au lieu de monter de +1 changer directement car = A puis continuer le for

Une fois ce patch appliqué faire les même test avec les fiches de test de ChiffreMin

1. Fonctionnalité testée :	On veut vérifier que la saisie s'arrête quand l'utilisateur saisit + de 80 caractère
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 A B D Z R → jusqu'à 80 caractère Le programme reconnaît quand + de 80 caractère sont entrée et s'arrête
3. Résultat attendu :	Le programme s'arrête quand il atteint 80 caractère
4. Résultat obtenu :	Le programme s'arrête à 10 caractères entrée
5. Conclusion :	Le programme s'arrête seulement à 10 caractère entrée car la constante MAX_NB_CAR est définit à 10. Si on change cette constante à 80, le programme marche et s'arrête lorsque la saisie atteint 80 caractères.

Fonction Déchiffrer

Pour réaliser la fonction déchiffrer on doit réaliser une fonction qui fait l'inverse de la fonction chiffrerCar. A l'instar de cette action paramétrée il faut aussi créer des sous fonction dechiffrerCarMaj et dechiffrerCarMin pour gérer des lettres majuscule et minuscule.

6. Action paramétrée `dechihiffrerCar()`

1. Fonctionnalité testée :	On veut vérifier qu'un caractère minuscule ou majuscule est déchiffré respectivement en majuscule ou minuscule
2. Scénario	

Donnée(s) d'entrée :	Z z B b
Situation(s) visée(s) :	L'action paramétrée dechiffrerCar fait la différence entre les caractère et les
3. Résultat attendu :	Les caractères majuscules sont déchiffrés en majuscule et les caractère minuscule en minuscule
4. Résultat obtenu :	Z z B b
5. Conclusion :	Le programme fonctionne correctement et les caractères déchiffrés sont les mêmes que ceux saisie de base.

BONUS :

Le but de la partie bonus est de chiffre cette fois ci des caractère spéciaux. Un problème évident se pose cependant, car contrairement au lettre majuscule et minuscule les caractères spéciaux donné ne se suivent pas dans la table ASCII. Par conséquent il est impossible de faire comme dans les programme précédent en rajoutant seulement la valeur de la clé au caractère.
Pour résoudre ce problème nous avons crée un tableau contenant tous les caractères spéciaux à chiffrer.

```
#define CAR_SPE { ',', ';', ':', '?', '!', '/', '+', '-', '*', '%' }
char tabSpe[10] = CAR_SPE;
```

Afin de chiffrer les caractère présent dans le tableau il nous fallait une fonction capable de repérer à quelle indice le caractère en entrée correspondait. Nous avons donc crée la fonction pos_tab(char car) qui retourne l'indice du tableau auquel le caractère se trouve.

```c
int pos_tab(char car)
{
    char tabSpe[10] = CAR_SPE;
    int i;
    int pos;
    for (i=0 ; i<10 ; i++)
    {
        if (car == tabSpe[i])
        {
            pos = i;
        }
    }
    return pos;
}
```

Avec cette fonction il ne nous restait plus qu'à coder 2 fonctions pour chiffrer et déchiffrer les caractère spéciaux

```c
char chiffrerSpe(char car, int cle)
{
    int index;
    int i;
    char tabSpe[10] = CAR_SPE;
    char carSpe;
    index = pos_tab(car);
    for (i=0 ; i<cle ; i++)
    {
        if (tabSpe[index] == tabSpe[9])
        {
            index = 0;
        }
        else
        {
            index = index + 1;
        }
        carSpe = tabSpe[index];
    }
    return carSpe;
}
```

```c
char dechiffrerSpe(char car, int cle)
{
    int index;
    int i;
    char tabSpe[10] = CAR_SPE;
    char carSpe;
    index = pos_tab(car);
    for (i=0 ; i<cle ; i++)
    {
        if (tabSpe[index] == tabSpe[0])
        {
            index = 9;
        }
        else
        {
            index = index - 1;
        }
        carSpe = tabSpe[index];
    }
    return carSpe;
}
```

Contrairement à chiffrerCar et DechiffrerCar ces fonctions n'incrémente pas le caractère directement mais plutôt l'indice du tableau puis affecte à la fin de la boucle for le caractère chiffré ou dechiffré à carSpe.

Il ne nous restait plus qu'à ajouter ces deux fonction dans chiffreCar et DechiffrerCar pour les implémenter dans le main

```
char chiffrerCar(char car, int cle)
{
    char carChiffre;

    if ((car >= 'a') && (car <= 'z'))
        {
            carChiffre = chiffrerMin(car, cle);
        }
    else if ((car >= 'A') && (car <= 'Z'))
        {
            carChiffre = chiffrerMaj(car, cle);
        }
    else
        {
            carChiffre = chiffrerSpe(car, cle);
        }
    return carChiffre;
}
```

```
char dechiffrerCar(char car, int cle)
{
    char carChiffre;

    if ((car >= 'a') && (car <= 'z'))
    {
        carChiffre = dechiffrerMin(car, cle);
    }
    else if ((car >= 'A') && (car <= 'Z'))
    {
        carChiffre = dechiffrerMaj(car, cle);
    }
    else
    {
        carChiffre = dechiffrerSpe(car,cle);
    }
    return carChiffre;
}
```

Fiches de test :

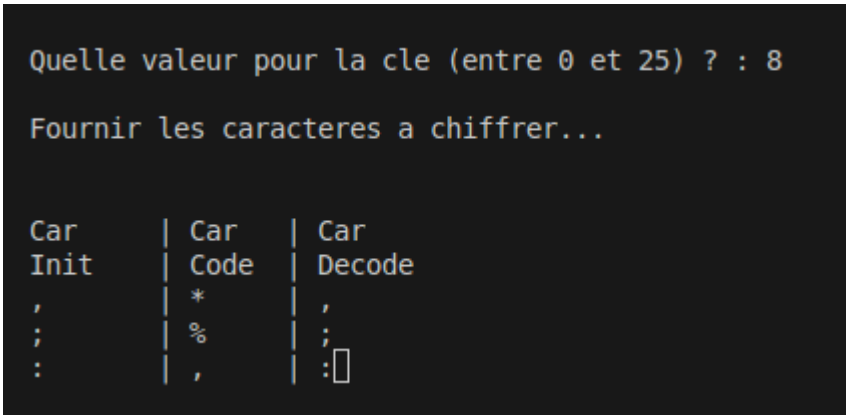
Action paramétrée `chiffrerSpe()`

Car Init	Car Code
,	!
;	/
:	+

1. Fonctionnalité testée :	On veut vérifier que les caractère spéciaux qu'on entre se chiffre correctement en fonction de la clé imposé
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 , ; : avec une clé de 4 L'action paramétrée chiffrerSpe chiffre correctement les caractère spéciaux entrée par l'utilisateur en fonction de la clé (4)
	, → ! ;→ / :→ +
4. Résultat obtenu :	

5. Conclusion :	Le programme fonctionne correctement et les caractères initiaux sont bien chiffrés selon la clé.

Action paramétrée `dechiffrerSpe()`

1. Fonctionnalité testée :	On veut vérifier que les caractère spéciaux qu'on entre sont déchiffrer correctement
2. Scénario Donnée(s) d'entrée : Situation(s) visée(s) :	 , ; : avec une clé de 8 L'action paramétrée chiffrerSpe déchiffre correctement les caractère spéciaux entrée par l'utilisateur et revoie les même caractère que ceux saisie initiallement
	, → , ;→ ; :→ :
4. Résultat obtenu :	
5. Conclusion :	Le programme fonctionne correctement et les caractères initiaux sont les même que les caractères déchiffrés.